GeomAlgo 模块

```
def nearZero(x):
    return math.fabs(x) < epsilon

def distance(obj1, obj2):
    if isinstance(obj1, Point3D) and isinstance(obj2, Line):    # Point Line
        pass
    elif isinstance(obj1, Point3D) and isinstance(obj2, Ray):    # Point Ray
        pass
    elif isinstance(obj1, Point3D) and isinstance(obj2, Segment):    # Point Segment
        pass
    elif isinstance(obj1, Point3D) and isinstance(obj2, Plane):    # Point Plane
        pass
    elif isinstance(obj1, Line) and isinstance(obj2, Line):    # Line Line
        pass
    elif isinstance(obj1, Line) and isinstance(obj2, Plane):    # Line Plane
        pass
    elif isinstance(obj1, Ray) and isinstance(obj2, Plane):    # Ray Plane
        pass
    elif isinstance(obj1, Segment) and isinstance(obj2, Plane):    # Segment Plane
        pass
    pass

def intersectLineLine(line1 : Line, line2 : Line):    计算两条直线的交点，返回：交点P、t1、t2
    pass                                              注意，如果交点P可能为None

def intersectSegmentPlane(seg : Segment, plane : Plane): 计算线段和平面的交点，返回交点P或None
    pass

def intersect(obj1, obj2):
    if isinstance(obj1, Line) and isinstance(obj2, Line):
        pass
    elif isinstance(obj1, Segment) and isinstance(obj2, Segment):
        pass
    elif isinstance(obj1, Line) and isinstance(obj2, Segment):
        pass
    elif isinstance(obj1, Line) and isinstance(obj2, Ray):
        pass
    elif isinstance(obj1, Ray) and isinstance(obj2, Segment):
        pass
    elif isinstance(obj1, Ray) and isinstance(obj2, Ray):
        pass
    elif isinstance(obj1, Line) and isinstance(obj2, Plane):
        pass
    elif isinstance(obj1, Ray) and isinstance(obj2, Plane):
```

```
            pass
        elif isinstance(obj1, Segment) and isinstance(obj2, Plane):
            pass
        pass

def pointOnRay(p : Point3D, ray : Ray):
    pass

# -1 : on polygon
# 1 : inside polygon
# 0 : outside polygon
def pointInPolygon(p : Point3D, polygon : Polyline):
    pass

def intersectTrianglePlane(triangle, plane):
    pass

def intersectTriangleZPlane(triangle, z):
    pass

def adjustPolygonDirs(polygons):
    pass

def rotatePolygons(polygons, angle, center = None):
    pass
```